

Trabajo Final — Opción E

Logistic Regression + LDA vs. Autoencoder + Logistic Regression

1. Introducción

Este trabajo final se basa en el artículo *A Deterministic Comparison of Classical Machine Learning and Hybrid Deep Representation Models for Intrusion Detection on NSL-KDD and CICIDS2017* (Arcos-Argudo, M., Bojorque, R. & Torres, A., 2025, *Algorithms*, 18(12), 749), en el cual se comparan clasificadores clásicos contra un pipeline híbrido que incorpora una componente de aprendizaje profundo, evaluados sobre el dataset CICIDS-2017.

El objetivo es que el estudiante replique de forma crítica y documentada la comparación entre dos clasificadores clásicos lineales (**Logistic Regression** y **Linear Discriminant Analysis**) y un pipeline híbrido de representación (**Autoencoder** + **Logistic Regression**), analice los resultados obtenidos y los contraste con los reportados en el paper (Tabla 6, régimen sin SMOTE).

2. Dataset

El dataset utilizado es el **CICIDS-2017** (Canadian Institute for Cybersecurity Intrusion Detection Evaluation Dataset 2017), disponible en:

<https://www.unb.ca/cic/datasets/ids-2017.html>

El dataset contiene tráfico de red capturado durante cinco días, incluyendo tráfico normal y distintos tipos de ataques, con aproximadamente 80 *features* numéricas de flujo. Para este trabajo se considera el problema en su versión **binaria**: clasificar cada flujo como tráfico normal (BENIGN) o ataque.

3. Preprocesamiento

Seguir los pasos de preprocesamiento de forma análoga al paper. El artículo ajusta todas las transformaciones **solo sobre el conjunto de entrenamiento** (sin fuga de información) y evalúa sobre el conjunto de test retenido.

Paso 1 — Tratar valores faltantes

Imputar los valores faltantes de las columnas numéricas con la mediana, ajustada únicamente sobre el conjunto de entrenamiento.

Paso 2 — Eliminar valores infinitos

Algunas columnas contienen valores $\pm\infty$ producto del cálculo de *features* de red. Eliminar o imputar las filas que los contengan antes del escalado.

Paso 3 — Consolidar etiquetas (binarización)

La columna *Label* contiene variantes textuales de los tipos de ataque con diferencias en mayúsculas, espacios y guiones según el archivo CSV de origen. Para el problema binario, mapear todas las etiquetas de ataque a una única clase “Attack” y conservar “BENIGN” como la clase normal:

- etiqueta “BENIGN” $\rightarrow$ 0 (normal)
- cualquier otra etiqueta $\rightarrow$ 1 (ataque)

Nota: usar `str.strip()` para normalizar espacios antes de comparar, ya que los CSV del CIC traen nombres y etiquetas con un espacio inicial.

Paso 4 — Escalado de *features*

Aplicar escalado min-max al rango $[0, 1]$ sobre las columnas numéricas, ajustando el escalador solo sobre el conjunto de entrenamiento y aplicándolo luego a *train* y *test*.

Paso 5 — Partición *train/test*

Separar en *train/test* con proporción 80/20, estratificada por la etiqueta binaria, usando una semilla fija (por ejemplo `random_state=42`) para reproducibilidad.

Aclaración: esta opción se trabaja en el régimen sin SMOTE, que es el que reporta la Tabla 6 del paper.

4. Marco teórico

Elaborar una explicación teórica de cada uno de los tres modelos: **Logistic Regression (LR)**, **Linear Discriminant Analysis (LDA)** y el pipeline **Autoencoder + LR (AE+LR)**. La explicación debe incluir:

- Intuición del modelo: ¿qué problema resuelve y cómo lo aborda?
- Para LR y LDA: el supuesto que hace cada uno y cómo construye su frontera de decisión lineal. ¿En qué se diferencian LR (modela directamente la probabilidad condicional) y LDA (asume distribuciones gaussianas por clase con igual covarianza)?
- Para el AE+LR: la arquitectura del autoencoder (capas del *encoder* y *decoder*, tamaño del espacio latente, funciones de activación) y el rol de cada componente.
- Fortalezas y limitaciones conocidas de cada enfoque (interpretabilidad, costo computacional, capacidad de capturar relaciones no lineales).

Punto conceptual: ¿cómo se usa un modelo no supervisado en un problema etiquetado?

El autoencoder **no se usa como clasificador final**. El pipeline correcto, y el que implementa el paper, es:

1. Entrenar el autoencoder de forma no supervisada, usando únicamente los *features* de los flujos (sin mirar la etiqueta).

2. Una vez entrenado, descartar el *decoder* y conservar solo el *encoder*.
3. Usar el *encoder* para transformar cada observación en su representación comprimida (*embedding*).
4. Entrenar una Logistic Regression **supervisada** sobre esos *embeddings*, que sí utiliza las etiquetas.

El autoencoder funciona como una etapa de *aprendizaje de representación* (ingeniería de variables automática), no como el clasificador en sí. Es conceptualmente análogo a aplicar PCA (Análisis de Componentes Principales) antes de un clasificador, con la diferencia de que el autoencoder puede aprender relaciones **no lineales** entre las variables, mientras que PCA solo captura relaciones lineales.

Bibliografía obligatoria:

- Para **LR** y **LDA**: Hastie, T., Tibshirani, R. & Friedman, J. — *The Elements of Statistical Learning* (<https://hastie.su.domains/ElemStatLearn/>). Ver Capítulo 4 (Linear Methods for Classification): Sección 4.3 (LDA) y Sección 4.4 (Logistic Regression).
- Para el **Autoencoder**: Goodfellow, I., Bengio, Y. & Courville, A. — *Deep Learning* (<https://www.deeplearningbook.org/>). Ver Capítulo 14 (Autoencoders).
- Arcos-Argudo, M., Bojorque, R. & Torres, A. (2025). A Deterministic Comparison of Classical Machine Learning and Hybrid Deep Representation Models for Intrusion Detection on NSL-KDD and CICIDS2017. *Algorithms*, 18(12), 749.

5. Implementación

Implementar los tres modelos en un Jupyter Notebook, siguiendo el pipeline de preprocesamiento descrito en la sección 3. El notebook debe ser **reproducible** y **auto-contenido**: cualquier persona debe poder ejecutarlo de principio a fin sin intervención manual, y las celdas de texto (markdown) deben explicar cada etapa.

1. Entrenar una **Logistic Regression** sobre los *features* originales preprocesados.
2. Entrenar un **Linear Discriminant Analysis** sobre los mismos *features* originales.
3. Entrenar un **Autoencoder** no supervisado sobre los *features* de entrenamiento (sin usar etiquetas). Documentar la arquitectura elegida.
4. Extraer los *embeddings* del *encoder* para *train* y *test*, y entrenar una segunda Logistic Regression sobre esos *embeddings* (pipeline AE+LR).

Para cada modelo, reportar sobre el conjunto de test: *Accuracy*, *Precision*, *Recall*, F1-score y AUC.

6. Evaluación y comparación con el paper

Contrastar los resultados obtenidos con los reportados en Arcos-Argudo et al. (2025), Tabla 6 (CICIDS-2017, sin SMOTE). Los valores de referencia son:

Modelo	Accuracy	Precision	Recall	F1-score	AUC
Logistic Regression (LR)	0,9878	0,9722	0,9783	0,9752	0,9961
LDA	0,9784	0,9426	0,9715	0,9569	0,9923
Autoencoder + LR	0,9859	0,9691	0,9738	0,9715	0,9960

Discutir brevemente las diferencias entre los resultados propios y los del paper, identificando posibles causas (versión del dataset, arquitectura del autoencoder, hiperparámetros, semilla aleatoria, etc.).

7. Análisis crítico

A diferencia de otros trabajos donde el modelo de aprendizaje profundo queda por detrás de los clásicos, acá el resultado es más sutil: los tres modelos alcanzan un rendimiento muy alto y muy parecido sobre CICIDS-2017, con AUC cercano al techo ($\approx 0,996$ para LR y AE+LR). Elaborar un análisis crítico respondiendo al menos:

1. ¿Qué modelo obtiene el mejor resultado en cada métrica? ¿Las diferencias entre LR, LDA y AE+LR son grandes o marginales?
2. Si el AE+LR no supera claramente a LR en este dataset, ¿qué justifica el costo adicional de entrenar un autoencoder? (Pista: pensar en el AUC y en la separabilidad, no solo en la accuracy.)
3. ¿Por qué un dataset tabular y bien separable como CICIDS-2017 favorece que un modelo lineal simple como LR alcance casi el techo de rendimiento?
4. ¿Qué diferencia hay entre LR y LDA en sus supuestos, y por qué pueden dar resultados tan parecidos en este caso?

8. ¿Cuándo gana el aprendizaje profundo?

El resultado del paper muestra que, en datos tabulares bien separables, el pipeline híbrido AE+LR apenas empata con una Logistic Regression simple. El objetivo de esta sección es identificar y demostrar un caso donde el aprendizaje de representación con autoencoder (o un modelo de aprendizaje profundo) aporte una ventaja **clara** sobre los modelos lineales.

Opciones (elegir una):

- **Opción A — Paper real:** buscar en la literatura un paper donde un autoencoder o un modelo de aprendizaje profundo supere claramente a los clasificadores lineales, en un dataset apropiado (imágenes, texto, señales, etc.). Citar el paper, describir el dataset y los resultados. Justificar por qué ese tipo de dato favorece al aprendizaje profundo.
- **Opción B — Toy example:** construir un dataset sintético donde las clases **no** sean linealmente separables (por ejemplo, dos espirales entrelazadas, o círculos concéntricos). Mostrar empíricamente en el notebook que LR/LDA fallan, mientras que el pipeline AE+LR (o una red más expresiva) logra separarlas gracias a la representación no lineal aprendida.

En ambos casos, justificar:

- ¿Qué características del dato o del problema favorecen al aprendizaje de representación no lineal?
- ¿Por qué los modelos lineales (LR, LDA) tienen dificultades en ese contexto?

- ¿Qué conclusión general se puede extraer sobre cuándo conviene cada tipo de modelo?

Esta sección debe estar presente tanto en el notebook (con código y análisis) como en las slides Beamer (con resultados y conclusiones).

9. Presentación Beamer

Preparar una presentación en LaTeX/Beamer que acompañe el notebook. La presentación debe tener entre **20 y 30 slides** y seguir la siguiente estructura:

1. **Portada.** Título del trabajo, nombre del estudiante, opción asignada, curso y fecha.
2. **Introducción al problema.** Motivación de la detección de intrusiones en redes. Presentación del paper de referencia y su pregunta central. ¿Por qué comparar modelos lineales contra un pipeline de representación?
3. **Descripción del dataset.** Qué es CICIDS-2017, cómo fue generado, qué tipos de ataques contiene. Planteo del problema binario (normal vs ataque).
4. **Explicación teórica de los modelos.** Una sección por modelo (LR, LDA y AE+LR). Incluir supuestos, fronteras de decisión, y la arquitectura del autoencoder. Explicar el punto conceptual del uso de un modelo no supervisado como etapa de representación.
5. **Pipeline de preprocesamiento.** Resumen visual de los pasos aplicados: imputación, valores infinitos, binarización de etiquetas, escalado y partición estratificada.
6. **Resultados obtenidos vs. paper.** Tabla comparativa de métricas propias vs. las del paper. Curvas ROC y matrices de confusión para cada modelo.
7. **Análisis crítico.** Discusión sobre por qué los tres modelos rinden casi igual en este dataset, y qué aporta el AE+LR en términos de separabilidad (AUC).
8. **¿Cuándo gana el aprendizaje profundo?** Presentación del paper encontrado o del toy example construido. Resultados comparativos y conclusión general.

La presentación debe entregarse como archivo .tex y .pdf. Debe ser auto-contenida: alguien que no haya visto el notebook debe poder comprender el trabajo leyendo solo las slides.

10. Estructura del repositorio (entrega en GitHub)

El trabajo se entrega como un repositorio de GitHub, siguiendo la estructura de proyecto vista en la Unidad 6 (Bloque 5: Portfolio profesional):

```
mi_proyecto/
  README.md
  data/
    raw/                (datos originales, nunca modificados)
    processed/          (datos limpios, listos para modelar)
  notebooks/            (numerados por orden de ejecución)
  src/
    utils.py             (funciones reutilizables)
  reports/              (la presentación Beamer compilada)
  requirements.txt
```

El `README.md` debe seguir las siete secciones mínimas vistas en clase: (1) título y descripción breve; (2) motivación; (3) datos (origen, tamaño, variables principales); (4) metodología (qué técnicas se usaron y por qué); (5) resultados principales; (6) cómo ejecutar; (7) limitaciones y próximos pasos.

Si los datos son grandes o sensibles, subir solo una muestra o las instrucciones para obtenerlos.

11. Entregables

- `tpfinal_mad1_lr_lda_vs_ae_lr.ipynb` — Jupyter Notebook completo y reproducible
- `tpfinal_mad1_lr_lda_vs_ae_lr.tex` — Código fuente de la presentación Beamer
- `tpfinal_mad1_lr_lda_vs_ae_lr.pdf` — PDF compilado de la presentación
- `tpfinal_mad1_lr_lda_vs_ae_lr_chat.json` — Conversación completa con la herramienta de IA utilizada. (NO SE SUBE AL REPOSITORIO, SINO QUE SE ENVIA POR MAIL)

12. Documentación del uso de IA

El uso de herramientas de IA (ChatGPT, Claude, etc.) está permitido durante el desarrollo del trabajo.

- Utilizar una **única sesión de chat** para todo el desarrollo del trabajo, de principio a fin.
- Al finalizar, exportar esa conversación completa y entregarla junto con el resto de los archivos. En ChatGPT: Configuración → Exportar datos, se entrega el archivo `conversations.json`. Alternativamente, el chat en texto plano.
- Nombrar el archivo como `tpfinal_mad1_lr_lda_vs_ae_lr_chat.json` (o `.txt`).

13. Criterios de evaluación

La mitad de la nota depende del **Jupyter Notebook**, evaluado en base a:

- Reproducibilidad: el notebook corre de principio a fin sin intervención.
- Auto-contenido: las celdas markdown explican cada etapa sin necesidad de consultar el paper.
- Claridad del código: organizado, comentado y sin celdas innecesarias.
- Correctitud de resultados: métricas comparables a las del paper.

La otra mitad depende de la **presentación Beamer**, evaluada en base a:

- Claridad: cada slide transmite una sola idea principal.
- Explicabilidad: los modelos quedan explicados para alguien que no los conoce.
- Auto-contenida: la presentación se entiende sola, sin el notebook.
- Calidad de la presentación: uso correcto del template, sin exceso de texto.

La organización del repositorio (estructura de carpetas y README) también se considera en la evaluación, en línea con lo visto en la Unidad 6.

A criterio del docente, se podrá solicitar una presentación oral como instancia de desempate o verificación.